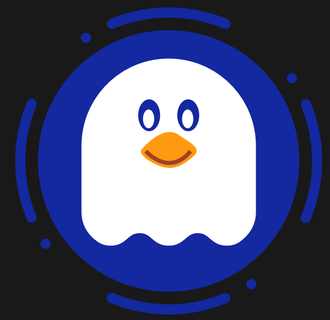
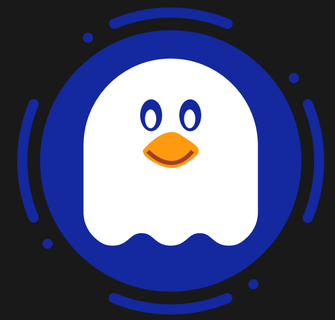


INTERFACING KERNEL C APIS FROM RUST

Kernel Recipes 2024

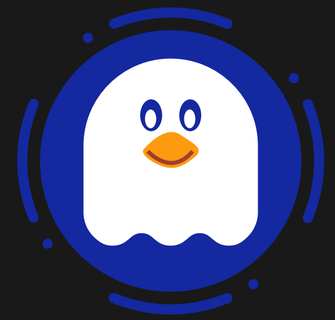
Andreas Hindborg

Samsung GOST



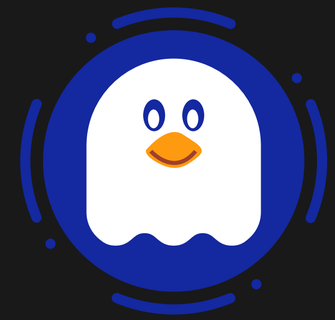
WHY SO COMPLICATED?

- Kernel Rust abstractions are somewhat **complicated**.
- **Where** does the complexity come from?



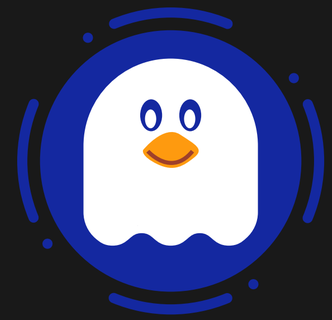
AGENDA 📄

- Memory Safety
- Block layer refresher
- Rust block device API



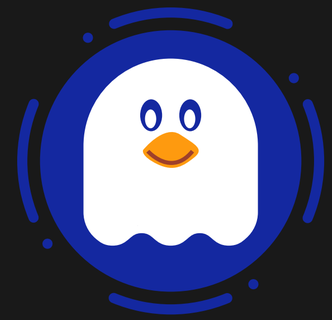
WHY CARE ABOUT MEMORY SAFETY 

- Microsoft: **70%** of all security bugs are memory safety issues [4]
- Chrome: **70%** of all security bugs are memory safety issues [5]
- **20%** of bugs fixed in stable Linux Kernel branches for drivers are memory safety issues [7]
- **65%** of recent Linux kernel vulnerabilities are memory safety issues [6]
- ASOP: Memory safety vulnerabilities **disproportionately represent** our most severe vulnerabilities [8]
- Google: Rust teams > **2x as productive** as C++ teams [1]
- Andreas: **41%** of fixes for `null_blk` are memory safety fixes [2]



MEMORY SAFETY BUGS 🐛

- Take time and **money** to fix
  - If we find them 😬
  - Need extensive up front testing and **fuzzing**
- Lead to **functional** problems
- Lead to **security** vulnerabilities
- Memory safe languages can **fix this** [3]



MEMORY SAFETY IN RUST 

Rust has a **safe** subset

- Memory safe
- Type safe
- Thread safe

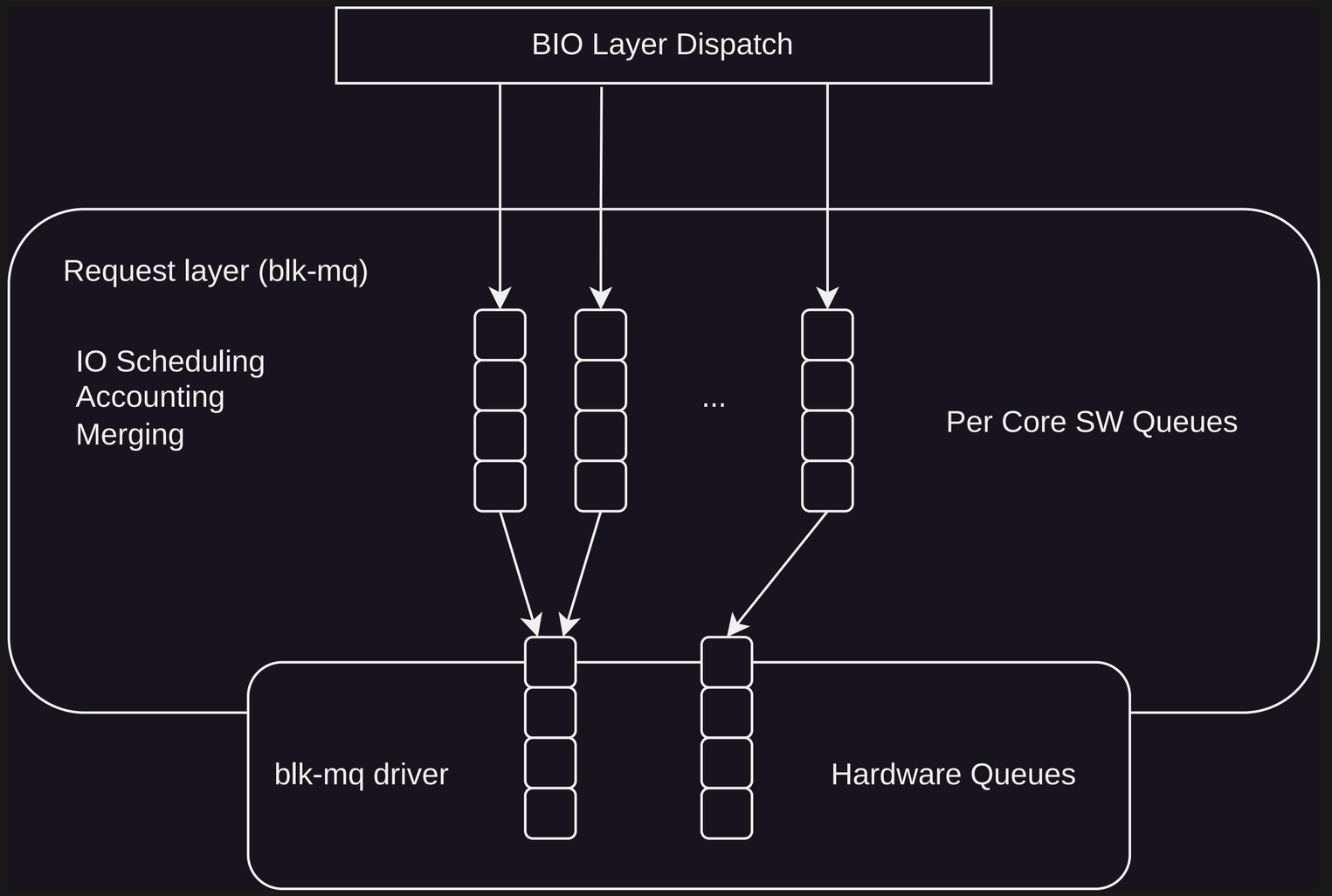
In **safe** Rust

- No buffer overflows
- No use after free
- No dereferencing null or invalid pointers
- No double free
- No pointer aliasing
- No type errors
- No data races



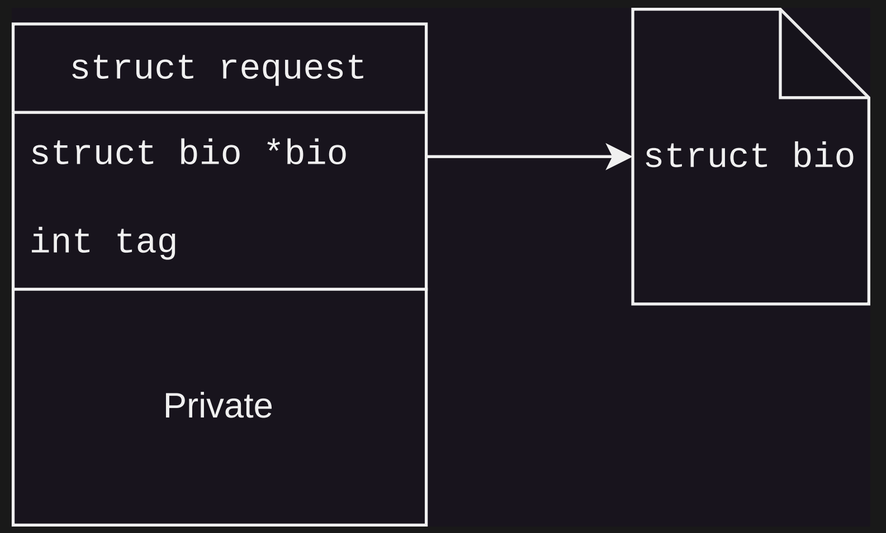
BLOCK LAYER REFRESHER

blk-mq

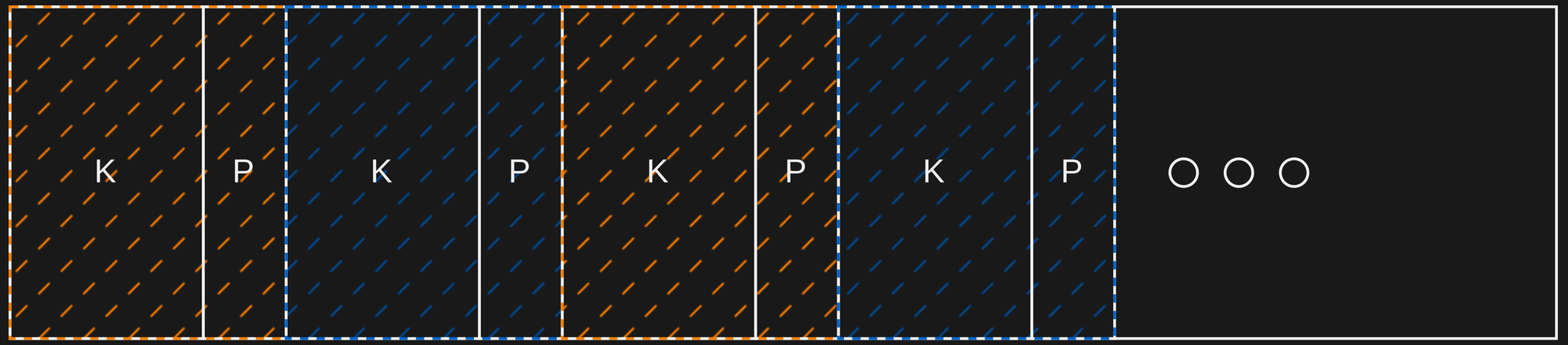


SAMSUNG

struct request



REQUEST CACHE



- Block layer allocates **array** of request structures
- **Block layer** initializes block layer part
- **Driver** initializes private area (`blk_mq_ops.init_request`)

C API



Kernel defines **vtable** for drivers to implement:

```
struct blk_mq_ops {
    blk_status_t (*queue_rq)(struct blk_mq_hw_ctx *,
                 const struct blk_mq_queue_data *);
    void (*complete)(struct request *);
    int (*poll)(struct blk_mq_hw_ctx *, struct io_comp_batch *);

    int (*init_request)(struct blk_mq_tag_set *set, struct request *,
                unsigned int, unsigned int);
    void (*exit_request)(struct blk_mq_tag_set *set, struct request *,
                 unsigned int);
    ...
}
```

Example from **null_blk**:

```
static const struct blk_mq_ops null_mq_ops = {
    .queue_rq   = null_queue_rq,
    .complete   = null_complete_rq,
    .poll       = null_poll,
    // No `init_request` or `exit_request` ?
    ...
};
```



INTERFACING FROM RUST



RUST: INITIALIZING `struct request`

SAMSUNG

We could do like C, but that would be **unsafe**:

```
pub trait Operations {
    type RequestData: Sized + Sync;

    unsafe fn new_request_data(rq: *mut Self::RequestData);
    ...
}
```

- References to uninitialized values trigger **Undefined Behavior**
- Writes to raw pointers are **unsafe**
- Rust values are movable
  - We should not be able to move out of `struct request`
  - We have to teach the compiler (`Pin`)

COMPLEXITY: `PinInit`



Instead we use `PinInit` to return an initializer for **in-place** initialization:

```
pub trait Operations {
    type RequestData: Sized + Sync;

    fn new_request_data() -> impl PinInit<Self::RequestData>;
    ...
}
```

- This adds complexity. PinInit is **~1K lines of code**, ~700 lines of docs.
  - But it prevents driver developers from messing up.
  - There is **no** performance cost.
- `PinInit` kernel docs
- In place initialization blog post

USAGE: SIMPLE




```
#[pin_data]
struct Pdu {
    #[pin]
    timer: kernel::hrtimer::Timer<Self>,
}

impl Operations for NullBlkDevice {
    type RequestData = Pdu;

    fn new_request_data() -> impl PinInit<Self::RequestData> {
        pin_init!( Pdu {
            timer <- kernel::hrtimer::Timer::new(),
        })
    }
    ...
}
```

Compiler checks: No uninitialized fields.

PLUMBING



Trait:

```
pub trait Operations {
    ...
    fn queue_rq(rq: &mut Request<Self>, is_last: bool) -> Result;
    fn poll() -> bool;
    ...
}
```

User implementation:

```
impl Operations for NullBlkDevice {
    ...
    fn queue_rq(rq: &mut mq::Request<Self>, is_last: bool) -> Result {
        ...
    }

    fn poll() -> bool {
        ...
    }
    ...
}
```

- We need to produce a **`struct blk_mq_ops`** vtable in a **memory safe** way

COMPLEXITY: RUST VTABLE

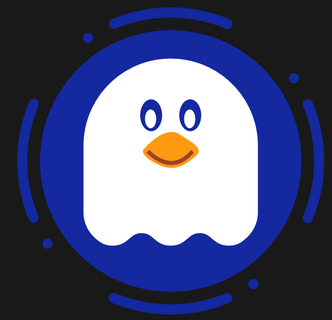



```rust
pub(crate) struct OperationsVTable<T: Operations>(...);

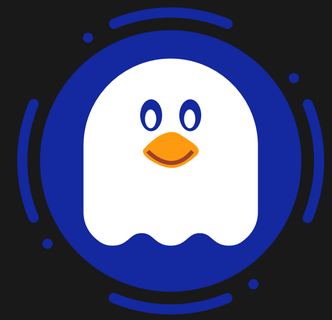
impl<T: Operations> OperationsVTable<T> {
    unsafe extern "C" fn queue_rq_callback(
        hctx: *mut bindings::blk_mq_hw_ctx,
        bd: *const bindings::blk_mq_queue_data,
    ) -> bindings::blk_status_t {
      // ... unsafe pointer manipulations and casts
      T::queue_rq(the_request)
    }

    const VTABLE: bindings::blk_mq_ops = bindings::blk_mq_ops {
        queue_rq: Some(Self::queue_rq_callback),
        complete: Some(Self::complete_callback),
        poll: if T::HAS_POLL {
            Some(Self::poll_callback)
        } else {
            None
        },
        init_request: Some(Self::init_request_callback),
        exit_request: Some(Self::exit_request_callback),
    };
}
```

INSTANTIATING THE VTABLE




```
impl<T: Operations> TagSet<T> {
    ...
    pub fn new(
    ...
    ) -> impl PinInit<Self, error::Error> {
        ...
        let tag_set =
            ...
                bindings::blk_mq_tag_set {
                    ops: OperationsVTable::<T>::build(),
                    ...
                };
    }
}
```



blk_mq_end_request

- Transition **ownership** of request from driver to block layer
- Calling this twice is potential **UAF**
- In C: Don’t call twice, that would be **stupid™**
- In Rust: It must be **impossible** to call twice



`blk_mq_tag_to_rq`

- In **completion context**, find the right `struct request`
- In C: Don’t pass a random tag to this function, that would be **stupid™**
- In Rust: It **must be safe** to pass a random tag to this function



SOLUTION: REFCOUNT `struct request`

```
@@ -2,3 +2,3 @@
     ...
-    fn queue_rq(rq: &mut Request<Self>, is_last: bool) -> Result;
+    fn queue_rq(rq: ARef<Request<Self>>, is_last: bool) -> Result;
```

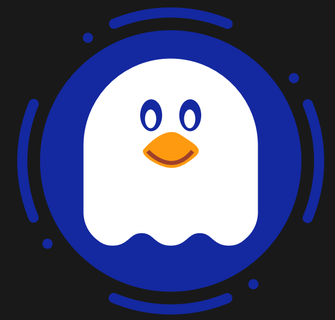
- 0: Owned by block layer
- 1: Owned by driver, no references
- 2: Owned by driver, 1 reference
- _: Owned by driver, > 1 reference



LET'S REUSE `request.ref`

```
struct request {
    ...
    atomic_t ref;
    ...
}
```

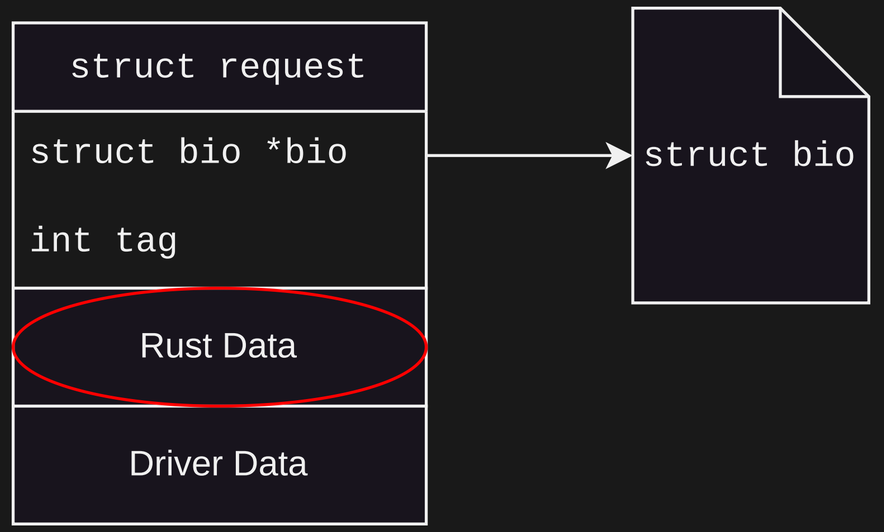
- Block layer **sets this to 1** when transferring ownership to driver
- What could go wrong?



WHAT COULD GO WRONG?

- iostat could be using `request.ref`
- Once in a while, things would break

ATTEMPT 2



- Store refcount in request **private data**
  - This adds **complexity**

FIX `Request::end`

- **Check refcount** in `Request::end`
- We want this to be the **last reference** in existence




```
impl<T: Operations> Request<T> {
    fn try_set_end(this: ARef<Self>)
      -> Result<*mut bindings::request, ARef<Self>> {
        // We can race with `TagSet::tag_to_rq`
        if let Err(_old) = this.wrapper_ref().refcount().compare_exchange(
            2,
            0,
            Ordering::Relaxed,
            Ordering::Relaxed,
        ) {
            return Err(this);
        }

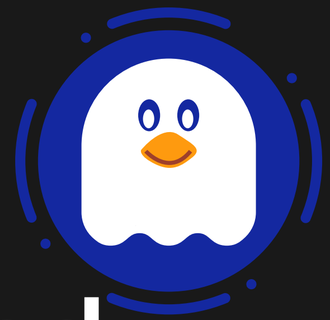
        let request_ptr = this.0.get();
        core::mem::forget(this);

        Ok(request_ptr)
    }
    ...
}
```

FIX `tag_to_rq`

- **Check refcount** in `tag_to_rq`

```
impl<T: Operations> TagSet<T> {
  pub fn tag_to_rq(&self, qid: u32, tag: u32) -> Option<ARef<Request<T>>> {
  ...
          refcount_ref.fetch_update(Ordering::Relaxed, Ordering::Relaxed, |x| {
              if x >= 1 {
                  Some(x+1)
              } else {
                  None
              }
          }).ok().map(|_| unsafe {Request::aref_from_raw(rq_ptr)})
      }
  }
  ...
}
```

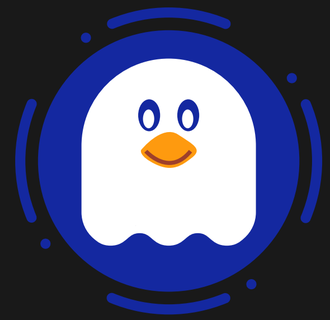


LOW USER OVERHEAD: Request::end

SAMSUNG

```
impl Operations for NullBlkDevice {
    #[inline(always)]
    fn queue_rq(rq: ARef<mq::Request<Self>>, _is_last: bool) -> Result {
        mq::Request::end_ok(rq)
            .map_err(|_e| kernel::error::code::EIO)
            // We take no refcounts on the request, so we expect to be able to
            // end the request. The request reference must be unique at this
            // point, and so `end_ok` cannot fail.
            .expect("Fatal error - expected to be able to end request");

        Ok(())
    }
    ...
}
```



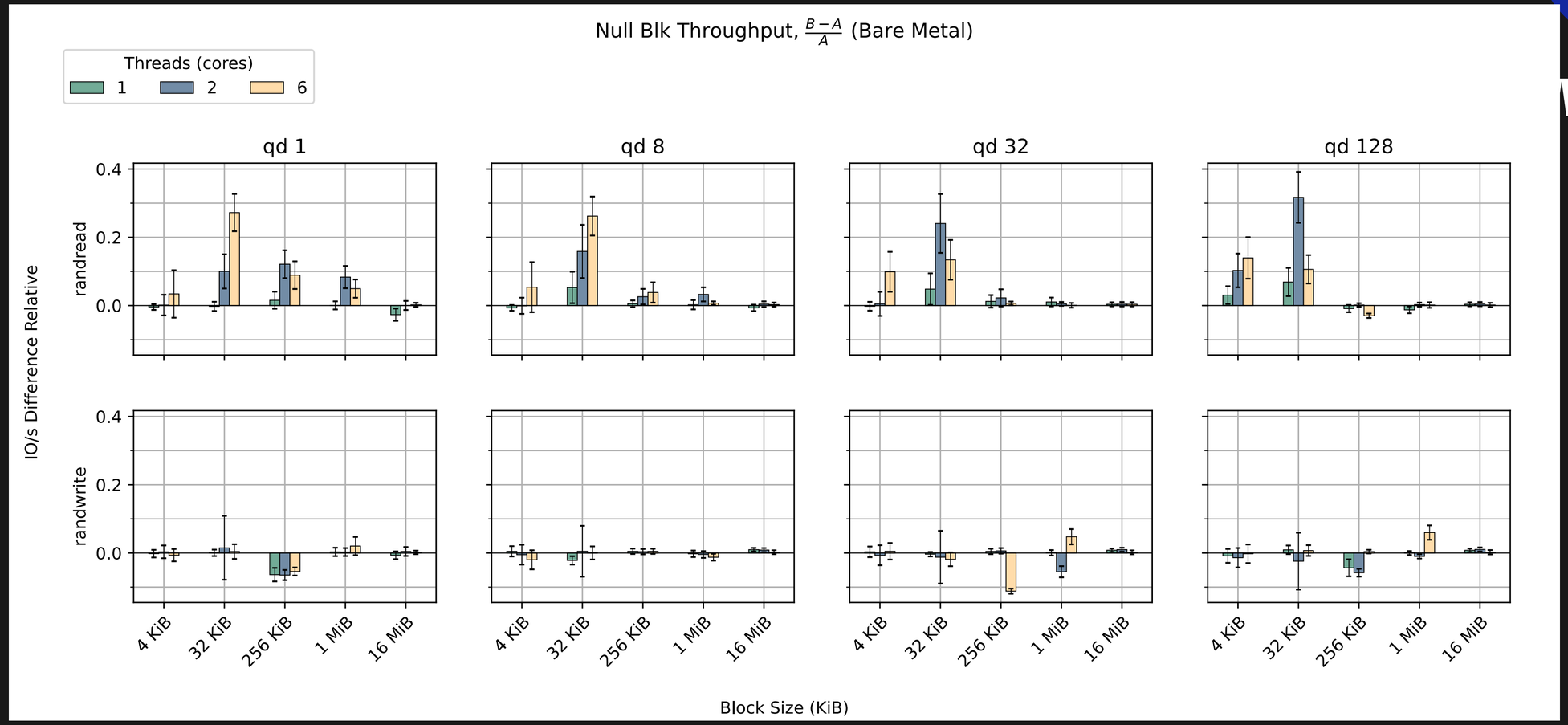
SAMSUNG

LOW USER OVERHEAD: `tag_to_mq`

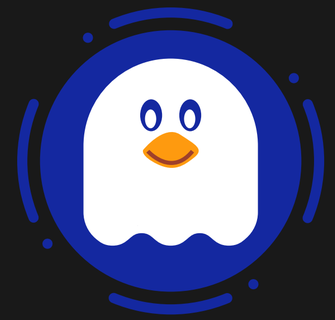
From Rust NVMe driver:

```
if let Some(rq) = self
    .tagset
    .tag_to_rq(...)
{
    ...
    kernel::block::mq::Request::complete(rq);
} else {
    let command_id = cqe.command_id;
    pr_warn!("invalid id completed: {}", command_id);
}
```

PERFORMANCE COST



Effect of disabling request refcount on `rnull-v6.11-rc2`. 40 samples for each configuration. Average change 2.0% improvement.



THE END

- Rust kernel abstractions are somewhat **complex**
- Complexity is in the abstractions
  - User is **spared**
  - APIs are is **simple**



REFERENCES

[1] Lars Bergstrom - Beyond Safety and Speed: How Rust Fuels Team Productivity.

[2] LKML: [LSF/MM/BPF TOPIC] blk_mq rust bindings.

[3] Memory Safe Languages in Android 13.

[4] *https://www.zdnet.com/article/microsoft-70-percent-of-all-security-bugs-are-memory-safety-issues/*.

[5] *https://www.chromium.org/Home/chromium-security/memory-safety/*.

[6] *https://lssna19.sched.com/event/RHaT/writing-linux-kernel-modules-in-safe-rust-geoffrey-thomas-two-sigma-investments-alex-gaynor-alloy*.

[7] A. A. Vasilyev, “Static verification for memory safety of Linux kernel drivers,” Proceedings of ISP RAS, 30:6 (2018), 143–160: *http://dx.doi.org/10.15514/ISPRAS-2018-30(6)-8*.

[8] Memory Safe Languages in Android 13: *https://security.googleblog.com/2022/12/memory-safe-languages-in-android-13.html*.